

Maintainability Index

Prof. Thiago Jabur

Maintainability Index

- MI
- Métrica de qualidade de código
- Indica Facilidade de manutenção

Índice de Manutenibilidade

- O MI é um indicador numérico que quantifica a saúde do seu código.
- Pense nele como um "check-up" médico para o software:
 - ele não diz exatamente onde está a doença, mas indica o quanto difícil será realizar uma cirurgia (manutenção) no futuro.

O que é

- Indicador numérico
- Avalia quão fácil é manter o código
- Objetivo: Oferecer uma visão rápida sobre a dívida técnica acumulada.
- Segundo Robert C. Martin (Clean Code), "a proporção de tempo gasto lendo código versus escrevendo é de mais de 10 para 1". O MI foca justamente em reduzir esse esforço de leitura e compreensão.

Origem

- Criado por Oman & Hagemeister
 - Universidade de Idaho
- Evolução: Embora criado nos anos 90, foi adaptado e normalizado (escala 0–100) para ferramentas modernas como o Visual Studio.

Fatores Avaliados

- **Complexidade Ciclomática (CC):** Mede o número de caminhos estruturalmente independentes através do código. Quanto mais if, for e while, maior a CC e menor o MI.
- **Volume de Halstead (HV):** Baseia-se no número de operadores e operandos exclusivos. Mede o "tamanho mental" necessário para entender a lógica.
- **Linhas de Código (LOC):** Simplesmente o tamanho do arquivo ou método. Métodos gigantes são naturalmente mais difíceis de manter.

Volume de Halstead (HV)

- As Métricas de Software de Halstead foram desenvolvidas por Maurice Halstead em 1977
- Segundo Halstead, "um programa de computador é a implementação de um algoritmo considerado como uma coleção de tokens que podem ser classificados como operadores ou operandos".
- Isso significa que o programa consiste em vários símbolos e elementos de dados que executam ações, operadores sobre operandos

Fórmula (conceito)

- Combinação das métricas
- Resultado normalizado (0–100)

Faixa de Pontuação	Classificação	Significado Prático
80 – 100	Alta	Código limpo, bem estruturado e fácil de evoluir.
50 – 79	Média	"Código Amarelo". Requer atenção; a manutenção começa a ficar lenta.
< 50	Baixa	Código legado ou "espaguete". Alto risco de bugs em cada alteração.

MI no Visual Studio

- A Microsoft utiliza uma escala de normalização que difere da fórmula acadêmica original de Oman & Hagemeister.
- Enquanto muitas ferramentas e a teoria original falam em uma escala de 0 a 100 , a Microsoft ajustou os limites para tornar a métrica mais pragmática para desenvolvedores modernos.

MI no Visual Studio

- $\Rightarrow 20$ - Highly Maintainable
- $\Rightarrow 10 \ \& \ < 20$ - Moderately Maintainable
- < 10 - Difficult to Maintain

Code Metrics Results

Filter: None

Min:

Max:

Hierarchy

▶

AccountController

▶

AccountController.ChallengeResult

▶

HomeController

▶

ManageController

▶

ManageController.ManageMessageld

▶

WebApplication2.Models

Maintainability Index

73

89

27

70

100

91

Cyclomatic Complexity

75

10

34

64

1

85

Depth of Inheritance

3

4

3

3

1

4

Class Coupling

55

8

12

58

0

21

Lines of Source code

470

29

390

379

11

218

Lines of Executable code

103

7

437

114

0

83

Error List

Output

Code Metrics Results

Maintainability Index:

64

Cyclomatic Complexity:

3

Halstead Volume:

323

Lines of Executable Code:

7

r()

■ 64

Interpretação

- Quanto maior, melhor
- Código mais fácil de manter

Faixas Comuns

- 80–100: Alta
- 50–79: Média
- <50: Baixa

Impacto na Qualidade

- **Redução de Bugs:** Códigos mais simples são menos propensos a erros ocultos.
- **Agilidade na Evolução:** Novas funcionalidades são implementadas mais rápido quando o código é legível.
- **Trecho de Referência:** No livro "Clean Code", Robert C. Martin afirma que o custo de manter um código ruim aumenta exponencialmente com o tempo, tornando o MC um guia essencial para evitar o colapso do projeto.

Como Melhorar

- Reduzir complexidade
 - Divida funções grandes em métodos menores
- Código limpo

Ferramentas

- Visual Studio: Fornece o cálculo nativo de métricas de código.
- SonarQube: Analisa o MI junto com débitos técnicos e vulnerabilidades.
- NDepend: Oferece visualizações avançadas e gráficos de tendência da manutenibilidade.

Conclusão

- Visão global da manutenibilidade
- Complementa outras métricas

Exemplo em java (MI < 50)

```
// MI BAIXO: Difícil de testar e entender rapidamente
public void processarPagamento(Fatura fatura) {
    if (fatura != null) {
        if (fatura.isPendente()) {
            if (fatura.getValor() > 0) {
                List<Item> itens = fatura.getItems();
                if (itens != null && !itens.isEmpty()) {
                    for (Item item : itens) {
                        // Lógica de cálculo complexa misturada com validação
                        double imposto = item.getPreco() * 0.15;
                        System.out.println("Processando: " + item.getNome());
                    }
                    fatura.setStatus("PROCESSADO");
                }
            }
        }
    }
}
```

Exemplo em Java (MI > 80)

```
// MI ALTO: Código limpo, linear e modular
public void processarPagamento(Fatura fatura) {
    if (!isFaturaValidaParaProcessamento(fatura)) {
        return;
    }
    aplicarImpostosAosItens(fatura.getItens());
    fatura.setStatus("PROCESSADO");
}

private boolean isFaturaValidaParaProcessamento(Fatura fatura) {
    return fatura != null
        && fatura.isPendente()
        && fatura.getValor() > 0
        && fatura.temItens();
}

private void aplicarImpostosAosItens(List<Item> itens) {
    itens.forEach(item -> {
        double imposto = calcularImposto(item);
        System.out.println("Processando: " + item.getNome() + " com imposto: " + imposto);
    });
}
```